

Real-Time Detection and Repair of LLM Agent Failures

Sunny Dubey
sjkumardube@gmail.com

*Manuscript draft. Every number in this paper is imported from a committed table in **results/**; Appendix A maps each section to its tables and merge commits. All results reproduce from clean checkouts at pinned seeds.*

Abstract

LLM agents fail mid-episode — they loop, cascade tool errors, drift off their goal, fabricate results, or silently absorb corrupted content — and the standard remedy, judging every step with a second LLM, costs more than the agent itself. We ask how much failure detection is achievable from *observable step telemetry alone* (semantic embeddings of step output, token-level uncertainty, action metadata), using monitors that cost microseconds per step and train only on healthy runs. Starting from a one-class echo-state-network (ESN) ensemble with CUSUM alarms, we build and validate, on 2,823 committed agent episodes across three frameworks, three local agent models spanning two families (qwen2.5 7b/3b, llama3.1 8b), and the gemini-2.5-flash API, a sequence of increasingly capable monitors: (1) the ESN, which wins decisively when failures have temporal room to develop (its advantage over a memoryless Mahalanobis baseline grows monotonically with post-onset horizon); (2) a calibrated ESN+Mahalanobis hybrid whose learned fusion weights track the deployment regime, and which lifts grand-mean AUROC above either parent taken alone (0.826 against 0.802 for the ESN and 0.807 for Mahalanobis) while sitting at or below the *per-dataset* better parent on 7 of 8 datasets in AUROC and on all 8 in detection rate — a pooled advantage, not a per-deployment guarantee; and (3) a content-grounding telemetry channel — nine causal features of tool-result content, including a 2-microsecond lexical relevance flag — that lifts the monitors’ shared blind spot (content corruption) from 0.28 to 0.59 pooled content detection (reaching 1.00 on the research collectors where the corruption is content-visible; honestly inert on frameworks whose corruption leaves result text unchanged), a +0.31 gain that holds at every seed, while behavioral detection improves rather than degrades (+0.045). Validation on organic (non-injected) failures reveals a taxonomy with an honest boundary: a deterministic five-line completion check catches failures of omission (7/7 silent aborts), the telemetry monitors transfer only *weakly* to failures of commission (1 of 3 fabrications) and rank the organic failure set at/below chance without recalibration, and plausible-value corruption requires external reference — the escalation layer’s job, for which our cost-optimal policy recovers 83% of judge-every-step detection at 8% of its calls against a *stipulated* judge, but only **44%** (0 of 5 seeds supported) once a real gemini-2.5-flash judge is measured and substituted; the call saving survives, the detection claim does not. The healthy null is specific to the (model, decoding-configuration) pair — cross-model transfer without recalibration falls to chance (AUROC 0.45–0.49) — and per-deployment calibration is label-free but not cheap in the way we previously claimed: measured against calibration budget, *ranking* is cheap (95% of full AUROC by ≈ 15 –50 healthy episodes) while the *operating point* is not (realized false alarms reach

twice the 5% budget only at $n \approx 15\text{--}48$, and on one deployment never do), so a deployment must verify its realized FA rather than trust the budget. Calibration is also itself weak on the hardest organic sets. Both burdens motivate a complementary layer that carries neither: deterministic verification, which recomputes a run’s stated total from the tool results that run actually received and confirms every required call was made, needing no null, no threshold and no per-deployment calibration. Head-to-head on the same episodes and labels it catches 60% of failures (96% with the coverage check) at **0 of 63 false positives** against the monitor’s 54% at 17%; it replicates on 120 later episodes at disjoint seeds scored frozen (54%, 93% with coverage, 0 of 64), transfers unchanged across model families to llama3.1:8b (**110 of 110 at 0 of 10**), and catches **26 of 26** provoked fabrications — the class on which a one-class monitor structurally cannot be scored, because provoking enough fabrication leaves too few healthy episodes to form a null. A third check validates tool results against the shapes their tool can return, which recovers part of the content-corruption blind spot the behavioural monitors leave: across every labelled corpus it flags **0 of 1825** healthy episodes while catching 46% of injected context corruption, 215 of 218 of them within one step of onset. Detection is then closed into repair: each flagged run is rolled back to its last fact-gathering step and re-run live, which recovers up to 45% of failures against a 16% resampling control and lifts net task success from **52% to 73%** for about one extra model call per run. The full system runs at $\sim 200\ \mu\text{s}$ per step, three orders of magnitude below a judge call.

1. Introduction

An agent episode is a sequence of steps; each step emits observable telemetry: what the agent said, how confident its tokens were, what tools it called, what they returned, how long everything took. The monitoring question is whether a lightweight, always-on watchdog over this stream can raise a calibrated alarm at derailment onset — steps before the task fails or the budget burns — without model internals, without labels, and without a second LLM in the loop.

Where this sits. Detecting failure while an episode is still running is no longer an unoccupied position. AgentForesight reframes failure attribution as online auditing — at each step the auditor sees only the prefix and must continue or alarm — and ships AFTraj-2K, 2,276 labelled multi-agent trajectories (arXiv:2605.08715); weakly supervised early alerting learns turn-level risk from trajectory-level labels alone and exposes an explicit accuracy/earliness dial (arXiv:2606.05414). Both use an LLM auditor. Separately, linear probes on hidden states predict eventual failure from the first interaction round, reported as substantially earlier than monitors restricted to observable behaviour (arXiv:2607.06503). We take that last result at face value: with the weights in hand, activations beat telemetry. So we claim neither mid-episode detection nor observable telemetry as novel. What we claim is the cost class and the deployment it serves — a one-class reservoir monitor at $\sim 200\ \mu\text{s}$ per step, fit on healthy runs only, with no LLM in the loop and no model internals, for the case where you are monitoring a hosted API or someone else’s agent and internals are not on offer. We have not evaluated on AFTraj-2K; those corpora are not built to supply the healthy-only null this method needs, and closing that gap is the next comparison rather than a claim we can make here.

This work answers that question constructively, and documents with equal care what such a watchdog cannot do. Contributions:

1. **A controlled testbed with ground-truth onsets** (failure injector, 43-dim telemetry) and an 11-monitor comparison under matched false-alarm budgets. The ESN-ensemble CUSUM channel-max monitor detects 0.71 ± 0.07 of failures at a 5% FA budget with a 4.6 ± 1.0 -step

mean budget saving, beating trained LSTM (0.61) and GRU (0.60) baselines by ~ 10 detection points over five dataset seeds (§4). With the credit assigned honestly: **the per-channel max-fusion wrapper, not the reservoir, carries most of that margin** — giving a GRU the same wrapper lifts it to det 0.76 / AUROC 0.873, past the ESN. The wrapper is the transferable contribution; the ESN stays the default for its false-alarm discipline (0.069 against 0.113) and $\sim 100\times$ faster fit, not because reservoirs are the only architecture that works here.

2. **Real-ecosystem validation at scale:** 2,823 episodes over 25 corpora, of which 770 use real tools (arXiv/Wikipedia/web/SQL/Python) with live agents (qwen2.5:7b, qwen2.5:3b, llama3.1:8b, gemini-2.5-flash) across bespoke, LangGraph, and AutoGen loops — with every trace committed and every table reproducible (§5).
3. **A diagnosis of when temporal monitoring pays:** the ESN’s detection advantage over a $50\times$ -cheaper Mahalanobis baseline grows monotonically with post-onset horizon (+9 points at ≤ 3 steps, +14 at 4–8, +40 at ≥ 9 ; $r = +0.25$ over 1,002 injected episodes) (§6).
4. **A calibrated hybrid** whose supervised fusion weights learn which regime a deployment is in (Mahalanobis weight share 0.34 on long-horizon data $\rightarrow$ 0.95 on short). Its advantage is a *pooled* one: grand-mean AUROC 0.826 beats either parent alone (ESN 0.802, Mahalanobis 0.807), because each parent collapses on some dataset and the fusion never does. Per dataset it is a different story — the fusion is at or below whichever parent is better there on 7 of 8 datasets in AUROC (mean -0.014) and on 8 of 8 in detection rate (mean -0.140 , driven by o1lama7b, 0.235 against the ESN’s 0.965). We claim robustness across deployments, not superiority within one (§7).
5. **A content-grounding telemetry channel** that closes the content blind spot: malformed JSON 0.07 $\rightarrow$ 0.93, off-topic retrievals 0.13 $\rightarrow$ 1.00 where applicable (via a binary lexical flag costing $2\ \mu s$), context corruption to the channel’s own ceiling — with behavioral detection *improved*, not traded (§8). A measured negative result: MiniLM embeddings ($2,200\times$ the cost) add nothing here.
6. **Fusion lessons with teeth:** single shared thresholds let one stream’s healthy tail price the other stream’s evidence out of the alarm budget (16/17 grounding detections destroyed, quantified); per-stream "dual-budget" thresholds with a train-max trip restore them, and are the only variant passing a strict never-degrade criterion at every seed (§7.3, §8.2).
7. **Organic-failure validation** and the commission/omission taxonomy: monitors trained on injected failures transfer only *weakly* to organic fabrication (1/3), are structurally blind to silent aborts (0/7) which a trivial completion check catches (7/7 at 0 FP), and the healthy null does not survive changes of sampling temperature or agent model — calibration is per-deployment, and cheap (§9).

2. Telemetry

Each step t emits x_t , built in four generations (all causal; later channels are strict supersets, and monitors selecting only earlier channels behave identically on wider episodes):

version	dims	content
v1 (base)	43	e_t: 32-dim hash embedding of step text; u_t: 4 token-surprisal aggregates (mean/max/slope/high-fraction); m_t: action one-hot, log latency, log output length, error flag
v2	43	tool RESULTS appended into step text (<code>[name({args}) -> result]</code>) so corrupted results reach the semantic channel
v3 (+x)	51	8 derived behavior dims: cosine drift, task-anchor similarity, tool success rate, retry count, per-tool latency, context ratio, reasoning depth, self-consistency
v4 (+g)	60	9 content-grounding dims: query $\leftrightarrow$ result and reasoning $\leftrightarrow$ result dissimilarity, result self-consistency (centroid, consecutive, EWMA-drift, 3-step memory), truncation-safe JSON-prefix validity, character-statistics anomaly, and a binary lexical relevance flag

The embedding is a deterministic char-3-gram feature hash (no model, no network); a MiniLM alternative exists strictly behind an explicit opt-in and is nowhere required (§8.4). v4 telemetry costs 608 μ s/step at the adapter (355 μ s for v3); the lexical flag alone costs 2 μ s/result.

3. Monitors

All monitors are one-class (fit on healthy episodes only) and causal (score_step sees `x_1..x_t`). Alarm thresholds come from healthy-validation episode maxima at a fixed FA budget (5% offline, 10% in the live demo).

- **ESN ensemble + CUSUM** (esn_cusum_max): per-channel reservoirs ($K=8$, 128 units) predict the next step; normalized surprise + ensemble disagreement feed a one-sided CUSUM; channels fuse by max. Fit in ~ 2 s; 189 μ s/step; 4 MB.
- **Baselines**: Mahalanobis and delta-Mahalanobis (Ledoit–Wolf, 4 μ s) — delta-Mahalanobis is a *strong* near-memoryless comparator ($\sim 51\times$ cheaper than the ESN, ahead of it on ≥ 1 real tool-cascade metric), not a straw-man — linear AR, and trained GRU/LSTM/TCN sequence models (fit 17–79 s). Cosine-drift and isolation forest are included as **negative controls**: on the synthetic study they detect at ≈ 0.10 and ≈ 0.07 (near the healthy-FA floor), bounding what a single-channel geometric drift and a static outlier model buy with no temporal state; they are not competitive baselines.
- **Hybrids** (§7): robust-z calibration of the ESN-CUSUM and delta-Mahalanobis streams against healthy medians/MADs, then weighted, max, gated, or logistic fusion; the logistic trains on ~ 20 labeled episodes (cross-fit, never scoring an episode its fold saw).
- **Grounded monitors** (§8): a memoryless per-dim robust-z max over the g channel joins as a third stream; the shipped variants are the content gate (behavioral score + boosted grounding override above the healthy-train maximum) and the dual-budget OR (per-stream thresholds).
- **Completion check** (§9): five lines, no learning — final step empty or a bare tool call $\Rightarrow$ task incomplete.

`derail.monitor.hybrid.recommended_monitor()` encodes the deployment policy: content gate when the g channel is present, 2-way weighted hybrid otherwise, supervised logistic when ≥ 20 labeled failures exist.

4. Controlled study

The simulator constructs the detector’s input telemetry directly (it writes the semantic, uncertainty, and metadata channels for each step — no LLM, tokenizer, or embedding measurement in the loop), so the class signatures are designed in and this study measures whether the monitors *recover* them; measured-telemetry evidence is the real-ecosystem study (§5). Headline (5 seeds, full size): the ESN detects **0.707 ± 0.068** of failures at a 4.4% realized FA with episode AUC **0.872 ± 0.015** and a mean survivorship-free lead of **4.6 steps**. H1 and H3b hold at four of five seeds and are honestly not supported / mixed at one; H2 and H3a hold at all five. The best non-ESN baselines: linear AR 0.607, LSTM 0.606, GRU 0.595. Confidence is calibrated by the healthy-score null — its healthy stream is uniform to $KS \approx 0.12$ (fused), and the oracle isotonic posterior (*with* labels) reaches ECE ≈ 0.03 (H3a). A cost-optimal escalation policy (operating point selected on calibration) recovers **83% of judge-every-step detection at 8% of its judge calls** (master seed; total-cost ratio 61%) (H3b).

This number is conditional on a judge we have now measured, and it does not survive. The judge in H3b is a stipulated noisy oracle (p_detect 0.90, p_false 0.02). Running a real gemini-2.5-flash judge on a labelled subset (172 distinct prompts, `judge_calibration_summary.json`) measures p_detect **0.548** (95% CI 0.44–0.65) and p_false **0.052** (0.020–0.13) — both stipulated values fall outside their measured intervals. Substituting the measured rates and re-running all five master seeds with everything else held fixed (`judge_sensitivity.csv`): the **call saving survives** (6% vs 8% of judge-every-step calls, cost ratio 0.63 vs 0.61) but detection recovery falls to **44% (range 39–48%)** and H3b is **SUPPORTED at 0 of 5 seeds**, against 4 of 5 for the stipulated judge. The escalation architecture still buys its cost saving; the *detection* claim at the published operating point should be read as 44%, not 83%.

5. Real-ecosystem validation

The committed corpus spans 2,823 episodes over 25 datasets: the v1 Gemini set (18), mock-tool framework sets (bespoke/LangGraph/AutoGen $\times$ two model generations, 774), the llama3.1:8b cross-family set (193), the lengthened Gemini set (125), and 770 real-tool episodes (10 real-task Gemini set; qwen2.5:7b research sets standard/long; qwen2.5:3b set; organic high-temperature set; organic serving-temperature set; demo sets). Failure labels come from a tool-layer injector with exact τ (7–8 classes including a prompt-layer goal hijack), except the organic sets: the research-task organic set is hand-labeled with per-episode evidence quotes, while the demo-task organic sets are labeled objectively by script from each run’s own tool results against a computable ground truth.

The two demo-task organic sets are seed-paired arms of one controlled comparison (pre-registered, $n=120$ each, identical tasks and model, differing only in sampling temperature — the failure-provoking 0.9 used by all earlier organic work versus the 0.2 actually served). Detection survives: organic arithmetic errors alarm at 46% against a 17% healthy false-alarm rate (Fisher $p=0.0025$, AUROC 0.733; 0.824 on a held-out corpus), and all-failure detection does not differ significantly between the arms (40% vs 54%, $p=0.12$), so the signal is not carried by high-temperature degradation. Reaching that conclusion required correcting the definition of *healthy*: runs that state the

right total but skip work the task specifies had been counted in the null, and the monitor separates those from genuinely healthy runs at AUROC 0.95–0.98, so carrying them inside the reference buried the signal entirely (under the old labels the same monitor measured at chance, AUROC 0.508).

Getting to that number required correcting two defects, both of which had inflated every earlier organic alarm rate. First, θ had been selected on the healthy episodes the monitor was fit on; with nested out-of-fold θ the realized false-alarm rate is 13% (T=0.9) and 18% (T=0.2) against a 10% budget. Second, a degenerate-scale defect (DESIGN.md Amendment 6) divided a telemetry dim carrying no healthy variation by a numerical floor instead of leaving it unscaled, so an uninformative channel became the most sensitive in the system: healthy episodes reached peaks of 1.0e9 against a corpus median of 0.46 and set the alarm threshold for everyone. Fixed at all five affected sites, the maximum healthy peak falls to 2.99; no committed study number moves, because the simulator has no degenerate dims and no held-out healthy episode in any real corpus deviates on one. What remains is genuine estimator variance on a heavy-tailed healthy distribution, not amplification. The dominant organic failure on this task — correct tool lookups followed by a wrong sum — is behaviourally silent, and is a verification problem rather than an anomaly-detection one.

The two layers report at different times because they answer different questions: anomaly is observable while an episode unfolds, whereas correctness does not exist until the agent commits to an answer. The monitor is therefore a causal step-by-step detector and the verification layer a gate between the agent and the user — a run failing its checks is rolled back and re-run before anything is delivered, so the end-to-end quantity of interest is task success rather than detection lead.

We therefore add a deterministic verification layer (`derail/verify/checks.py`, DESIGN.md Module 8) that recomputes a run’s stated total from the tool results that run actually received and confirms every required call was made. It reads only what the agent observed — never the world the task was generated from, so it is strictly weaker than our oracle and deployable as-is — and it needs no null, no threshold and no per-deployment calibration, which is precisely the recalibration burden §9 documents for the one-class monitors. Head-to-head on the same episodes and objective labels, at the served temperature it catches 60% of failures (96% adding the coverage check) at **0/63 false positives**, against the monitor’s 54% at 17%: comparable recall, but the checks raise no false alarm and need no calibration corpus. Because the checks were written by inspecting failures in that arm, we validate them on a further 120 episodes collected afterwards at disjoint task seeds and scored frozen: 54% (93% with coverage) at **0/64 false positives**, with arithmetic errors again caught 36/36. The zero false-positive rate and the arithmetic result both replicate.

The checks transfer across model families. A llama3.1:8b arm collected at the same serving temperature on the **same 120 task seeds**, so both models price identical worlds, and scored with the checks frozen and nothing tuned for it, catches **110/110 failures at 0/10 false positives**; llama fails far more often on this task (110 of 120 runs) but every failure it produces is caught. That test also exposed a real defect: `total_consistency` had required the stated total to equal the sum of *every* observed price, so an agent that prices six flights for a four-leg tour and correctly totals the right four was called wrong. It now asks whether some selection of the declared size reproduces the total, which admits an unused lookup while a dropped or double-counted one still leaves no valid selection.

Fabrication, which every earlier study reported as underpowered against a pre-registered floor of ten events, is now testable: a provoked corpus carries 26 labelled fabrications and the checks catch 26/26. That corpus cannot score the behavioural monitor at all — provoking enough fabrication

leaves two healthy episodes against the fifteen a null requires — which is the asymmetry in one line: a check can be evaluated exactly where a one-class monitor structurally cannot.

The checks are complementary by construction: a run that prices three of four legs and totals exactly those three is self-consistent, so only coverage catches it. This is the honest division of labour the organic evidence implies — trajectory monitoring for behavioural derailment, deterministic checking for silent incorrectness.

Checking the evidence, not only the answer. A third check inverts the direction of the other two. Each tool declares the shapes a successful result may take — `lookup_flight` returns a price or a no-route message and nothing else — so a result matching none of them was malformed at the boundary, and the agent should never have been handed it. Because it compares against a declared shape rather than an answer, it reports at the step the result arrives, which is earlier than any other signal in the system can exist. Scored across every labelled corpus (`tool_contract_coverage.csv`) it flags **0 of 1825 healthy episodes** — the property that lets it ship with no null — while catching **46%** of injected context corruption and 44% of looping, whose injector induces the loop by returning a retry message no tool contract admits. Every other injected class sits at **0%**, so this is a contract check and not a general-purpose alarm, and where it fires it is immediate: 215 of 218 flagged episodes are caught within one step of onset.

This recovers part of the content blind spot of §8 by a different route, and its limit is the same one stated there, drawn at the tool instead of the answer: it is silent on corruption that keeps a legal shape. A price altered from \$361 to \$605 is a well-formed price, and separating it from a real one still requires the external reference of §9.

Detection that changes the outcome. The study so far established that failures can be detected, not that detection helps. We close that loop (`derail/intervene/`, DESIGN.md Module 9): each flagged episode is rolled back to the last fact-gathering step and re-run live, paired on the identical prefix and task, with success graded by the objective labeller against a ground truth the repair prompt never sees. Every rate below is the mean of three independent repeats of each retry, with the observed range. Rolling back and merely resampling recovers 16% (15–18%) — retry luck is real, and is the control every other rung must beat. Three repair prompts clear it: naming the failing check without values recovers **45%** (44–47%, $p=0.0005$), undirected "re-check your work" 36% ($p=0.0347$), and naming the check with the recomputed values 36% ($p=0.0192$), lifting net task success over all 120 episodes from **52% to 73%** for about one extra model call per run. Two rungs do not beat the control: withholding the prompt when merely completeness is at fault (21%, $p=0.61$) and directing the agent to re-add the figures with its calculator (28%, $p=0.17$). No policy damaged a correct run, because the checks flagged no already-correct episode.

That last rung is a negative result worth stating, because it was the most promising hypothesis available. The dominant failure is arithmetic over figures the agent looked up correctly, and the agent holds an unused calculator, so directing the retry to a tool that cannot make an arithmetic error should have dominated every prompt that merely asks the model to think again. It does not: at 28% it is below undirected re-checking and does not separate from retry luck at this sample size. Whatever the repair prompts are buying, it is not arithmetic offloading.

Asking for a re-check is therefore what works, and naming the failing check works best. One comparison in that set is worth isolating: `total_consistency` derives the total from the agent's own figures, so for a run that merely mis-added, that value is the correct answer — 26 of 55 such prompts contain it. The variant that names the failing check and no value at all contains it in 0 of 55 and recovers at least as much, so the improvement does not come from handing the agent its

answer.

The cost is measured rather than assumed — extra calls from the study’s own rows, per-step latency from the retried traces. The repair fires on 55 of 120 runs and adds 2.8 calls and about 7.5 s to a flagged run, buying one recovered failure per ~6 model calls; amortised over every run, including the 65 never flagged, that is ~1.3 extra calls and ~3.4 s.

On real traces the ESN remains the best single temporal monitor (Gemini v1: det 0.71 at AUROC 0.840, against LSTM’s 0.68 at 0.752), though on that corpus both sit at a 20-27% realized false-alarm rate over 15 healthy test episodes, and a 5% budget is unreachable from its 16 validation episodes by an empirical quantile. Two structural findings reframe the problem: monitors do not transfer across frameworks (off-diagonal AUC 0.06–0.74 vs 0.64–1.00 in-domain, the extremes contributed by the 18-episode Gemini corpus, which carries a single positive and is degenerate at both ends), and — decisively for what follows — the memoryless delta-Mahalanobis *wins* on short-episode datasets (research7b: 0.839 vs 0.784 AUC).

The commercial-API evidence rests on a lengthened corpus. The original Gemini set (**real**) is 18 episodes with a single positive and a mean length of 5 steps against a washout of 3 — too short to carry a labelled detection claim, and we do not make one from it. We therefore re-collected Gemini on the *same* long research task, injection classes, onset and step budget as the qwen **real_research7b_long** corpus, so the two differ only by provider: **real_gemini_long** is **125 episodes with 65 positives across 7 classes** (mean T 8.8; 54 positives have post-onset horizon ≥ 3). Scored, it reaches **AUROC 0.794** (ESN) / **0.824** (hybrid-max) — the monitors work on a frontier API. Detection at a matched FA budget is initially lower than on the paired qwen corpus (**0.38 vs 0.57** at FA 0.17) while AUROC is essentially identical (0.794 vs 0.790), and two candidate explanations were tested rather than assumed.

The first — that this Gemini tier **rejects response_logprobs**, so all 125 episodes lack the token-surprisal channel the qwen corpus has — is **refuted**. Ablating that channel *within* four corpora that do carry it (same episodes, splits, seeds and thresholds; **telemetry_dependence.csv**) changes nothing: mean ESN effect **AUROC +0.000, detection +0.002**. Token logprobs are close to inert on real traces, so their absence cannot explain the gap — and the deployment requirement is correspondingly weaker than we had stated.

The second — post-onset **horizon** — explains it. The Gemini positives are much shorter after onset (median 4 vs 8 steps; 28% at horizon ≤ 3 against qwen’s 2%), and the ESN needs post-onset steps to integrate evidence (§6). Restricting both corpora to positives with horizon ≥ 4 (**telemetry_horizon_matched.csv**) closes the gap: Gemini **0.53** detection at AUROC **0.902**, qwen **0.56** at 0.785. The provider difference was a horizon difference, and at matched horizon the commercial API is not harder to monitor than the local model.

5.7 External validation: AFTraj-2K

Every number above comes from corpora we built, so none of them can say whether the benchmark is hard. AFTraj-2K (arXiv:2605.08715, CC-BY-4.0) is someone else’s: 2,276 multi-agent trajectories over math, coding and agentic domains, with an earliest-decisive-error step annotated by consensus of LLM judges. **derail.experiments.import_aftraj** converts it into our trace format — agent turns become steps, an environment turn is folded into the step that issued the call, and **mistake_step** becomes tau. Nothing about the monitors changes. 1,882 of the trajectories survive the same $T \geq 4$ filter the other corpora use: 1,111 healthy, 771 failures. The corpus carries no token logprobs,

so this runs on **e+m+x** — the u channel is not exercised at all.

monitor	AUROC	detection	FA	$\mu\text{s}/\text{step}$
esn_cusum_max	0.745	0.048	0.023	173
hybrid_weighted50	0.760	0.047	0.018	177
hybrid_gated	0.754	0.048	0.018	176
hybrid_logistic	0.665	0.034	0.014	—
delta_mahalanobis	0.616	0.022	0.032	3.7

Ranking transfers zero-shot. AUROC 0.745 on another project’s agents, frameworks and tasks, with the uncertainty channel absent entirely, against 0.802 pooled on our own. Nothing was re-trained or tuned: the monitors are fitted on AFTraj’s own healthy runs and scored as-is.

The horizon law predicted the operating point, and held. §6 claims detection needs post-onset runway to accumulate evidence. AFTraj tests that claim directly, on failures annotated by someone else:

horizon	n	ESN	Mahalanobis	ESN – Maha
0	51	0.000	0.000	0.000
1–3	423	0.007	0.009	−0.002
4–8	244	0.029	0.016	+0.012
≥ 9	53	0.509	0.170	+0.340

correlation +0.365 over 771 episodes. Where a failure has room to develop the ESN detects **half** of them and beats the memoryless baseline by 34 points; where it does not, neither monitor detects anything and the ordering between them is noise. Only **53 of 771** AFTraj failures (6.9%) sit in that regime — 61% end within three steps of their onset — so pooled detection is **0.048**.

That pooled figure is the mechanism’s prediction, not an excuse offered after seeing it: §6 fixed the relationship between horizon and detection on our corpora, and AFTraj supplies a failure distribution concentrated where the relationship says detection is impossible. A law that forecasts its own failure region and then survives a corpus it never saw is stronger evidence than a higher pooled number would have been. It also bounds deployment precisely: this monitor is for failures that develop over several steps, and AFTraj is mostly not that.

One asymmetry is worth recording: on failures AFTraj’s judges *diagnosed* in uncorrupted runs detection is 0.114, against 0.004 on their *injected* corruptions — the monitor is far better on naturally occurring failures than on synthetic ones, the opposite of the direction an injection-trained method is usually accused of.

Against the auditor. AgentForesight-7B reports 66.44 Exact-F1 on this corpus with a 7B LLM auditing every step, at roughly a thousand times the cost per step. Detection at a false-alarm budget is not Exact-F1 and the two do not belong in one column, so we compare the axis that is commensurable: 173 μs and no model call, against one 7B forward pass. The claim this section supports is that ranking transfers zero-shot to a foreign ecosystem at three orders of magnitude less compute, and that the regime where it also alarms is predictable in advance.

5.8 A second external corpus: ATBench

AFTraj is one benchmark and could be idiosyncratic. ATBench (arXiv:2604.02022, Apache-2.0) is a second, from a different group, with a different idea of what going wrong means: 1,000 trajectories labelled safe or unsafe under a taxonomy of risk source, failure mode and real-world harm. 604 have the four agent steps our filter requires (279 safe, 325 unsafe). It labels *trajectories*, never steps, so there is no tau — lead, delay and the horizon diagnosis are undefined here and are not reported. Detection still is: an alarm anywhere in the run, against a threshold picked on held-out safe runs at the 5% budget.

monitor	AUROC	detection	FA
esn_cusum_max	0.779	0.311	0.071
delta_mahalanobis	0.457	0.268	0.161
hybrid_weighted50	0.463	0.277	0.179
hybrid_max	0.463	0.277	0.161
hybrid_gated	0.438	0.268	0.179

The ESN alarms here: detection **0.311** at AUROC 0.779, against 0.048 on AFTraj. ATBench’s unsafe behaviour is a *pattern of action* — an over-privileged call, an unvalidated tool result — rather than a single wrong value, and a behavioural monitor is built for exactly that.

This is consistent with the horizon law but is **not a test of it**, and should not be read as one: ATBench labels whole trajectories and never says which step went wrong, so there is no onset, no post-onset horizon, and nothing to regress detection against. §5.7 is the law’s only out-of-sample test. What the two corpora jointly show is narrower and still worth having — the monitor fires on foreign data when the failure has the shape the mechanism assumes, and does not when it does not.

The fusion’s limit is confirmed independently. §7.2 argues the hybrid’s advantage is pooled rather than dominant; ATBench shows the failure mode that implies. Mahalanobis lands at 0.457 here, and every 50/50 blend goes with it — 0.438 to 0.463, against the ESN’s 0.779 alone. The hybrid buys robustness when the regime is unknown and costs heavily when a parent is not merely weaker but uninformative, so a deployment resembling ATBench should run the ESN alone. That is a conclusion we could state from our own data and can now show on someone else’s.

Per failure mode the split is sharp, and it lands exactly on the blind spot §8 already documents:

failure mode	n	detection
unconfirmed or over-privileged action	59	0.508
incorrect tool parameters	20	0.500
failure to validate tool outputs	55	0.473
insecure interaction or execution	24	0.333
unauthorized information disclosure	29	0.276
flawed planning or reasoning	26	0.192
procedural deviation or inaction	19	0.105
tool misuse in a specific context	22	0.045
inaccurate or misleading information	26	0.038

Failures that change what the agent *does* are caught around half the time; failures that only change what it *says* are not caught at all. That is the same content blind spot the grounding channel was built for (§8), confirmed on labels we did not write.

Two limits on how hard this can be read: the held-out safe set is 56 episodes, so the AUROC is directional rather than tight, and the realized false-alarm rate of 0.071 overshoots the 5% budget in the way §5.5 describes for small validation sets. We also discarded one number in analysis — an AUROC restricted to safe runs that themselves contained a handled risk looked much stronger, but only about five such episodes reach the test split, so it measures nothing.

6. When does temporal monitoring pay? The horizon diagnosis

Per-episode analysis over 1,002 injected episodes across eight datasets (`hybrid_diagnosis.csv`): the ESN–Mahalanobis detection gap is **+0.09** when the post-onset horizon ($T-1-\tau$) is ≤ 3 steps, **+0.14** at 4–8, and **+0.40** at ≥ 9 (correlation +0.25). The ESN’s CUSUM needs steps to integrate evidence; the Mahalanobis distance fires on the first anomalous step or never. Read the direction carefully: averaged over episodes the ESN never *loses* a horizon band, its margin merely collapses to almost nothing when there is nothing to accumulate. Where Mahalanobis wins is at the *dataset* level, on the corpora made almost entirely of short-horizon episodes (`real_research7b` 0.848 vs 0.777 AUROC, `real_research3b` 0.665 vs 0.556). A controlled replication (`research7b_long`: identical tasks/tools/ τ , 10 tool calls instead of 5) moves ESN detection $0.27 \rightarrow 0.60$ and lead $0.23 \rightarrow 3.64$ steps — horizon is a binding constraint, though the memoryless baseline also benefits from more chances to fire.

7. The calibrated hybrid

7.1 Mechanism, not just metrics

Of 1,002 injected episodes, 372 are detected by both parents, **273 by the ESN only, 57 by Mahalanobis only**, and 300 by neither — **330 of 1,002 (32.9%)** are visible to exactly one detector. The cross-fit logistic fusion recovers **100% / 66% / 79%** of the both/ESN-only/Maha-only cells and fires on just **1%** of the cases neither parent flags. Its learned Mahalanobis weight share tracks the regime: 0.38 (simulator, long horizons) $\rightarrow$ 0.99 (short-horizon research sets). In the 2-D calibrated-score plane the decision boundary rotates from horizontal (ESN decides) through diagonal to vertical (Mahalanobis decides) across datasets (Figure 1, `results/figures/hybrid_explain.png`).

7.2 Stability and generalization

Five seeds (reservoirs and fold assignment varied, splits held fixed): grand AUROC logistic **0.830** $\pm$ **0.006** $>$ max 0.817 $>$ Mahalanobis 0.812 ± 0.003

gated $0.810 \approx$ weighted $0.810 >$ ESN 0.800 ± 0.004 ; the logistic–parent

difference is positive at **every** seed (min +0.025 vs ESN, +0.010 vs Maha — the advantage is if anything larger after the evaluation fixes). Note that Δ -Mahalanobis now grand-means *above* the ESN and the weighted/gated hybrids: the memoryless baseline is strong, and only the *learned* logistic fusion beats it. The per-dataset cross-seed variance is mixed — fusion tightens it several-fold on the ESN’s noisiest real set (`real_research7b`, $0.009 \rightarrow 0.002$) but not on the short 3b/long sets

— so we claim a consistent mean advantage, not a variance guarantee. On a held-out framework (LangGraph, never used during development) the best hybrid stays within CI of the local winner and significantly above the local loser, though the logistic’s edge narrows there.

What the pooled mean hides. The grand mean is an unweighted average over eight datasets, and it flatters the fusion: the logistic wins it because *each parent collapses somewhere* (the ESN at 0.556 on real_research3b, Mahalanobis at 0.294 detection on ollama7b) while the fusion never collapses. Compared instead against whichever parent is better *on that dataset*, the fusion is at or below it almost everywhere (`results/tables/hybrid_benchmark.csv`):

dataset	ESN	Δ -Maha	better parent	logistic	Δ
sim	0.890	0.786	0.890	0.889	−0.000
autogen7b	0.833	0.774	0.833	0.777	−0.056
langgraph7b	0.828	0.885	0.885	0.884	−0.002
ollama7b	0.994	0.895	0.994	0.982	−0.013
real_research3b	0.556	0.665	0.665	0.643	−0.022
real_research7b	0.777	0.848	0.848	0.847	−0.001
real_research7b_long	0.790	0.849	0.849	0.857	+ 0.008
gemini	0.749	0.756	0.756	0.731	−0.025
grand mean	0.802	0.807	0.840	0.826	−0.014

Table: Episode AUROC. The fusion is below the per-dataset better parent on 7 of 8 datasets; it is above on one (real_research7b_long).

dataset	ESN	Δ -Maha	better parent	logistic	Δ
sim	0.780	0.378	0.780	0.725	−0.055
autogen7b	0.511	0.386	0.511	0.341	−0.170
langgraph7b	0.505	0.736	0.736	0.714	−0.022
ollama7b	0.965	0.294	0.965	0.235	−0.729
real_research3b	0.348	0.130	0.348	0.261	−0.087
real_research7b	0.374	0.415	0.415	0.398	−0.018
real_research7b_long	0.571	0.690	0.690	0.667	−0.024
gemini	0.709	0.582	0.709	0.696	−0.013
grand mean	0.595	0.452	0.644	0.505	−0.140

Table: Detection rate at the operating threshold. The fusion is below the per-dataset better parent on all 8, and the ollama7b cell (0.235 against 0.965) is a collapse, not a rounding difference — the learned weights put 0.99 share on Mahalanobis there, which is the wrong parent for that set.

So the honest statement is: **the hybrid is the best choice when you do not know which regime you are in, and the wrong choice when you do.** A deployment that can identify its own regime should run that regime’s parent. This is a robustness result, not a dominance result, and the "matches or beats both parents at every seed" framing of earlier drafts conflated a per-seed grand mean with per-dataset performance.

7.3 The fusion lesson

Under a val-quantile threshold only episode-max *ordering* matters, so a shared threshold lets the stream with the heavier healthy tail set the bar for both. Measured: 16 of 17 grounding-only context-corruption detections were destroyed by a single behavioral-tail healthy episode. No monotone rescaling can fix an ordering problem; per-stream thresholds can, and small validation sets forbid naive budget splits (2.5% of 24 episodes rounds to $\theta = \max$). The working construction: full budget on the behavioral stream, a healthy-train-maximum trip on the grounding stream (~zero FA spend), immediate override for the binary lexical flag.

8. Closing the content blind spot

8.1 The grounding channel

Behavioral and statistical monitors share one blind spot: corruption that changes *data* without changing *behavior* (context corruption, wrong documents, malformed JSON all ≤ 0.30 detection). The g channel adds nine causal content features (Table, §2), each engineered against a documented failure of a prior attempt — notably a JSON validity check that is truncation-safe by prefix-completion (storage truncation is never flagged) and a binary lexical relevance flag: a document-like result sharing **zero** content words with both its query and the task. The flag separates almost perfectly (28/28 wrong-document episodes, 3/330 healthy) because continuous coverage leaves healthy tangential documents a hair from decoys, which per-dim normalization cannot hold apart; and it self-disables in domains where healthy training episodes ever trip it (one-class clean-null gating).

8.2 Results under the success criterion

The criterion: content classes must improve, behavioral classes must not degrade. Pooled over 874 injected episodes across ten datasets (`grounding_multiseed_criterion.csv`, mean over five seeds):

fusion	content (n=313)	behavioral (n=561)
content gate	0.278 → 0.592	0.746 → 0.791
adaptive	0.278 → 0.539	0.746 → 0.783
logistic+g (labels)	0.278 → 0.322	0.746 → 0.713

Per class: malformed_json 0.12 → 0.82; context_corruption 0.28 → 0.37; wrong_document 0.12 → 0.12 (the lexical dim helps only in the datasets whose decoys are lexically off-topic, so the pooled mean shows no gain) on every clean-null dataset (and 1.00 everywhere for the supervised logistic); context corruption 0.18 → 0.43 pooled and to the channel’s own ceiling where content is visible (research7b 0 → 0.64–0.73 vs channel 0.77; research7b_long 0 → 0.83 = ceiling). Multiseed: content gains never fall below +0.29 at any seed for any grounded fusion, but only **dual-budget** holds behavioral non-degradation at every seed (min exactly 0.000) — the strict-guarantee deployment.

8.3 The prompt-hijack class

Goal drift injected realistically — an instruction inside a tool result redirecting the agent — is detected at **0.91** by the grounded gate vs 0.09 by the ESN alone: a fifth class where the con-

tent channel decides. (The simulator’s synthetic slow-rotation drift remains undetectable by every monitor at budget, 0.0125, and is documented as by-construction hard.)

8.4 The measured negative result

MiniLM embeddings for all semantic dims (4.4 ms/text, 5.7 s load, ~90 MB, vs 207 μ s hash and 2 μ s lexical): context corruption on the one unreachable dataset stays at exactly 0.00, and behavioral AUROC there *degrades* (0.82 $\rightarrow$ 0.70). That residual — semantically plausible value corruption — is reference-free undetectable: no consistency check against the episode’s own content can flag values that look ordinary. Detecting it requires external reference (re-query, cross-source, or a judge), i.e., the escalation layer (§4), not better telemetry.

9. Organic failures and the limits of the paradigm

Thirty non-injected episodes at the model’s natural temperature (0.9 vs the curated 0.2) yield 11 organic failures: 7 silent aborts, 3 fabricated counts (e.g., `len()` of a string reported as "262 papers"), 1 off-topic retrieval blended into an answer. Scored by monitors carrying the calibration they were deployed with on the deployed calibration (all numbers from `organic_validation.csv`, and not refit on this data):

- **Commission transfers only weakly:** the anomaly monitors catch just **1 of the 3** fabrications — failure modes never present in training, and the honest transfer is partial, not the clean sweep an earlier draft claimed.
- **Omission is invisible — but trivially fixable:** 0/7 aborts alarm (a quiet empty step is the opposite of anomalous activity); the deterministic five-line completion check catches all 7. Worse, the anomaly scores rank this whole organic failure population *at or below* chance against healthy episodes (within-organic AUROC 0.31–0.42) — the quantitative face of the omission blindness, and a caution against over-reading organic transfer.
- **The healthy null is (model, decoding)-specific:** swapping the agent model (7b $\rightarrow$ 3b, same tasks/tools) collapses transferred monitors to **at chance** (AUROC 0.45–0.49, FA 1.00); in-domain recalibration on 3b’s own healthy split recovers only to AUROC 0.56–0.67 (best: delta-Mahalanobis 0.67). Per-deployment calibration is mandatory — and cheap, since it is label-free — but on this hard organic 3b set even recalibration is weak.
- **The same result across a model FAMILY, without the 3b confound** (`model_transfer_family.csv`): llama3.1:8b was collected on the identical task/tool/injector plan as the qwen ollama7b corpus (193 episodes accepted of 380 attempts; mean episode length 6.8 vs 6.1, so this is not a length effect). Calibrated on itself, llama8b reaches **AUROC 0.885** (ESN; 0.893 delta-Mahalanobis) at 0.12 healthy FA — the monitors are **not qwen-specific**. Calibrated on qwen and deployed on llama with no refit, the same monitors sit at **chance (AUROC 0.527)** with a 0.75 false-alarm rate — its 0.91 "detection" is alarm-on-everything, not detection. This is the cleaner statement of the recalibration requirement: in the 7b $\rightarrow$ 3b study one could argue the 3b target was simply hard to monitor (in-domain 0.56), whereas here the *same* target scores 0.885 when calibrated on itself and 0.527 when calibrated on another family. Recalibration is a property of the calibration, not of the target.

Combined coverage on the organic set is **8/11** (7 from the completion check + 1 fabrication from the monitors); the remaining escapes are the 2 fabrications the monitors miss and the

reference-free-undetectable retrieval blend. A dedicated preregistered fabrication study (`organic_hallucination.csv`, 55 demo-task episodes) is explicitly **underpowered** — only 2 genuine hallucinations arise (preregistered minimum 10), at a 36% healthy false-alarm rate — so it makes **no detection claim**. The organic paradigm is thus a documented *limit*, not a headline result: the monitors’ value is on the injected/real studies and the escalation layer, not on unrecalibrated organic transfer.

10. Deployment guidance and live system

- **Default:** `recommended_monitor()` — the grounded content gate on v4 telemetry (label-free); dual-budget serving when a hard never-degrade-behavioral guarantee is required; supervised logistic fusion when ~20 labeled failures exist.
- **Calibrate per deployment:** same model, same decoding config, ~30 healthy episodes, no labels.
- **Ship the completion check** beside the monitor (omission failures).
- **Escalate, don’t guess**, for plausible-value corruption (83% of judge coverage at 8% of judge calls).
- **Cost:** ~204 μ s/step monitor-side (content gate, mean over six datasets; `results/tables/grounding_benchmark.csv`) plus ~0.7 ms/step telemetry extraction (v4 grounding path, hash-embedding dominated; measured by `verification/telemetry_benchmark.py -> results/tables/telemetry_runtime.csv`). Both are wall-clock and machine-sensitive; the whole stack stays ~3 orders of magnitude below one judge-LLM call.

The live demo serves exactly this architecture (real qwen2.5:7b agent, five injection buttons, per-factor explainability including grounding factors). Two live lessons hardened its calibration protocol. (1) Its first rehearsal, under a single shared threshold, missed three of five classes — the §7.3 lesson reproduced live before dual-budget serving fixed it. (2) A falsification control (a no-op hijack: task "rewritten" to identical text, so only benign follow-up probes differ from a healthy run) showed that a healthy null containing no probe-extended runs alarms on the probe pattern alone ~9 steps into probing — which had inflated earlier late-probe "detections", including a context-corruption +8 that does not survive the corrected null. The healthy corpus therefore includes probed healthy runs. (3) Wall-clock latency features were **neutralized** in the demo monitor (symmetrically at calibration and serving): on a shared local machine they measure the machine, not the agent — a 2.4 $\times$ inference slowdown drove 3/3 healthy runs to latency-led false alarms. Removing them left every detection margin *larger* (hallucination peak 1.48 $\rightarrow$ 4.04, hijack 2.09 $\rightarrow$ 4.27), since healthy maxima were no longer inflated by machine noise. Cloud deployments, where latency is stationary infrastructure, retain these features; the study tables are unaffected.

(4) The demo’s agent was given the shared generic tool suite, which includes a `search_catalog` tool irrelevant to pricing a trip. It contaminated both the runs and the reference: 46/113 corpus runs called it, and a catalog price corrupted the final total in 15 of 26 wrong bills (the agent variously added it or multiplied the trip subtotal by it) — while the monitor stayed silent, because with the misuse present in ~40% of the *healthy* reference it was inside the learned normal. This is the sharpest illustration of the one-class trade-off in the paper: a systematic error that pervades the reference becomes undetectable by construction. Scoping the agent’s tools to its task and re-collecting the null removed it (catalog calls 46/113 $\rightarrow$ 0/80; correct bills 51% $\rightarrow$ 70%, the residual

being genuine model arithmetic errors).

Rehearsal of record (corrected null, machine-invariant monitor, task-scoped toolset; 80-run corpus, $\theta_b10 = 11.85$): healthy $\times 2$ clean (peaks 0.38/0.31); looping +0, goal hijack +1, tool cascade +1, hallucination +10, context corruption +8 — **0 missed, 0 false alarms**. The tighter null (θ_b10 16.12 $\rightarrow$ 11.85) lifted the two weakest classes over the line. The corpus has since been extended to 120 runs, of which the three declared exclusion policies of §5 retain 58, and θ is recomputed from it at every startup rather than pinned; served through the rolling baseline of Module 10 it reports **trusted** at $n=58$ with a realized false-alarm rate of 8.6% against its 10% budget. The demo also runs the verification and repair layers live: a run whose checks fail is rolled back and re-run before its answer is shown. We explicitly do *not* claim corruption is solved *behaviourally*: it cleared by 0.15 in a single run, and the 26-episode E2E matrix over three injection timings measured it at 2/4. What closes the button in the live system is the tool-contract check of §5, which rejects the garbled result at the corrupted step rather than waiting for behavioural evidence that this world’s terse results cannot supply; a fresh headless rehearsal reports it at onset+0 against the behavioural monitor’s onset+12. The E2E matrix also shows *late* hallucination injection is undetectable by construction — the agent has already gathered the real data, so answering "from memory" produces a grounded, verified-correct answer.

11. Limitations

1. **Injected $\neq$ organic**, mostly: labels (except §9) come from an injector with fixed signatures; the lexical flag’s perfection against four fixed decoys is the cautionary example. Organic validation is small ($n=30$, single labeler with auditable evidence).
2. **Plausible-value corruption is out of reach** for any reference-free monitor (measured through hash and MiniLM alike). Corruption that *destroys* a result’s declared shape is not: the tool-contract check catches 46% of the injected class at zero false positives, which bounds this limitation to the plausible-value half rather than the whole of it.
3. **Synthetic slow goal drift** remains undetectable at budget for all monitors; only its realistic prompt-hijack variant is solved.
4. **Short-horizon failures** bound temporal detection: timeout detection is 0.50 on 5-step episodes and 1.00 on 11-step ones — horizon, not monitor, is the limit.
5. **No transfer without recalibration** across frameworks, agent models, or decoding configurations; the mitigation (cheap one-class recalibration) is validated, universality is not claimed.
6. **Small validation sets** quantize thresholds (realized FA 0–17% at a 5% budget per dataset); several per-class cells have $n=6$ –22.
7. Trained sequence baselines (GRU/LSTM) were tuned no further than the fairness diagnostics of the base study; the comparison bar is matched budgets, not exhaustive architecture search.

12. Conclusion

Observable step telemetry, scored by microsecond-scale one-class monitors, covers most of the agent-failure space: temporal behavioral failures (ESN, when horizon exists), abrupt state failures

(Mahalanobis), content corruption (grounding channel + lexical flag), and silent aborts (a completion check) — with a calibrated hybrid that learns which regime it is in, never trades the classes it already had, and explains its alarms. What remains genuinely out of reach — semantically plausible corruption — is precisely characterized and priced: it is the escalation layer’s job, at 8% of the always-judge cost. Where a run’s answer can be recomputed from what the agent observed, a deterministic check is the stronger instrument: it needs no null and no calibration, raises no false alarm on this task, transfers across model families unchanged, and can be evaluated on failure classes too rare to leave a one-class monitor a null at all. Detection is worth having only if it changes the outcome, and here it does — rolling a flagged run back to its last fact-gathering step and re-running it lifts net task success from 52% to 73% for roughly one extra model call. The engineering constant throughout is honest calibration: every threshold from healthy data of the exact deployment, every fusion respecting per-stream nulls, every claim from a committed, reproducible table.

Appendix A — provenance map (claims → artifacts)

Every claim below is computed from a committed file, and `BASELINE_MANIFEST.json` records a SHA-256 for each one, so a reader can check that a number in the text came from the file in the repository (`py -m devtools.artifact_manifest -check` recomputes them all).

Claim	Artifact	Regenerate with
Real per-class coverage	<code>17b_per_class.csv</code>	<code>run_hybrid_study</code>
Cross-family transfer	<code>model_transfer_family.csv</code>	<code>run_model_transfer</code>
gemini-2.5-flash corpus	<code>gemini_long*.csv</code>	<code>run_hybrid_study</code>
Measured judge	<code>judge_calibration_summary.json</code>	<code>run_judge_calibration</code>
Judge consequence for H3b	<code>judge_sensitivity.csv</code>	<code>judge_sensitivity</code>
Telemetry-channel ablation	<code>telemetry_dependence.csv</code>	<code>telemetry_dependence</code>
Recalibration cost	<code>recalibration_cost.csv</code>	<code>recalibration_cost</code>
Statistical power	<code>power_analysis*.csv</code>	<code>power_analysis</code>
Adversarial limit	<code>adversarial_evasion.csv,</code> <code>tamper_check.csv</code>	<code>tamper_check</code>
Organic + provoked fabrication	<code>organic_hallucination*.csv,</code> <code>provoked_fabrication.csv</code>	<code>score_provoked_fabrication</code>
Checks vs monitor, head to head	<code>verification_vs_monitor.csv,</code> <code>verification_cold.csv</code>	<code>derail.verify.run_verification_study</code>
Frozen holdout and llama transfer	<code>verification_holdout.csv,</code> <code>verification_organic_llama8b_cold.csv</code>	<code>derail.verify.run_verification_study</code>

Claim	Artifact	Regenerate with
Tool-contract coverage	<code>tool_contract_coverage.csv</code>	<code>derail.verify.run_verification_study</code> <code>-contract-coverage</code>
Repair policies and cost	<code>repair_policies.csv</code>	<code>derail.intervene.evaluate_repair_policies</code>
Alarm-triggered repair and escalation	<code>alarm_repair.csv</code>	live demo episodes, five classes x five seeds
Simulator study	<code>multiseed*.csv</code> , <code>h1_*</code> , <code>h3_*</code>	<code>run_multiseed</code>

Run each as `py -m <package>.<module>`: study runners live under `derail.experiments`, analyses under `experimental` and `verification`.

Appendix B — reproducibility protocol

One idea = one `exp/*` branch from `main`; merge requires (a) paired statistics (McNemar / sign-flip permutation / Wilcoxon, bootstrap Δ AUC CIs), (b) byte-exact reproduction of all result tables from a clean checkout at pinned seeds (timing columns excluded), (c) no regression of published tables (ungrounded paths re-verified byte-exact at each grounding-era merge). All randomness flows through `rng_for` streams; installing optional packages never changes results (explicit opt-ins only). Every published number is regenerated from the committed code and data, and the seed-7 replication reproduces exactly.